

Full Name	Imra Tanveer
Internship Domain	Python Development
Submission Type	Month 1 Tasks 1-2 and Month 2 Tasks 3-4

This report summarizes the completed Python development projects, implementation approach, features, and sample working output.

1. Introduction

Python development involves writing, testing, and maintaining applications using Python. This report presents four beginner-friendly console applications created as part of the Arch Technologies Python Development internship. The projects demonstrate control flow, functions, lists, dictionaries, input validation, random number generation, and basic object-oriented programming.

2. Project Summary

Month	Task	Project	Main Concepts
Month 1	Task 1	Dice Rolling Game	Random numbers, loops, scoring
Month 1	Task 2	To-Do List Application	Lists, dictionaries, progress bar, menu system
Month 2	Task 3	Number Guessing Game	Random numbers, hints, validation, attempts
Month 2	Task 4	ATM Simulation	OOP classes, deposits, withdrawals, balance checks

3. Task 1: Dice Rolling Game

Objective

The Dice Rolling Game simulates rolling a pair of dice. Each roll generates two random values between 1 and 6, displays the result, calculates a score, and asks the user whether to roll again.

Key Features

Uses the random module to generate dice values.

Calculates total score based on doubles and high rolls.

Uses a loop so the player can roll multiple times.

Displays the final score when the player exits.

Implementation Explanation

The program is organized with a roll_dice function for random dice generation, a score_roll function for scoring rules, and a main function for user interaction. This keeps the logic clean and easy to test.

Sample Working Output

Die 1: 4 Die 2: 6 Total: 10 Great roll! You earned 3 points. Current score: 3 Final score: 3

4. Task 2: To-Do List Application

Objective

The To-Do List Application allows users to add tasks, view tasks, mark tasks as done, remove tasks, and see progress through a text progress bar.

Key Features

Stores tasks as dictionaries containing title and completion status.

Displays all tasks with Done or Not Done labels.

Includes a progress bar based on completed task percentage.

Validates task numbers before updating or removing tasks.

Implementation Explanation

The program uses a list of dictionaries to manage task data. Helper functions build the progress bar, display tasks, and validate task numbers. The menu loop lets the user manage tasks until choosing Exit.

Sample Working Output

Your Tasks: 1. Complete Python report - Done 2. Practice coding - Not Done Progress: [#####-----]
50% Completed 1 of 2 tasks.

5. Task 3: Number Guessing Game

Objective

The Number Guessing Game asks the computer to select a random number between 1 and 100. The player guesses the number, and the program gives Too High or Too Low hints until the player wins or runs out of attempts.

Key Features

Generates a random secret number between 1 and 100.

Allows seven attempts per round.

Provides hints after every incorrect guess.

Handles invalid non-numeric input safely.

Implementation Explanation

The `check_guess` function compares the guess with the secret number and returns feedback. The `play_round` function manages attempts and hints, while `main` allows repeated games.

Sample Working Output

Enter your guess: 50 Too low. Attempts left: 6 Enter your guess: 75 Too high. Attempts left: 5 Congratulations!
You guessed it in 3 attempts.

6. Task 4: ATM Simulation

Objective

The ATM Simulation project creates a simple banking system where users can check balance, deposit money, and withdraw funds. It demonstrates basic object-oriented programming concepts.

Key Features

Uses an `ATMAccount` class to encapsulate account balance and operations.

Uses an `ATMController` class to handle menus and user input.

Validates deposit and withdrawal amounts.

Prevents withdrawals that exceed the available balance.

Implementation Explanation

The account logic is separated from the user-interface logic. `ATMAccount` handles balance changes, while `ATMController` manages menu choices, input reading, and displaying messages.

Sample Working Output

Enter amount to deposit: \$1000 \$1000.00 deposited successfully. Enter amount to withdraw: \$250 \$250.00
withdrawn successfully. Current Balance: \$750.00

7. Code Files Included

`task1_dice_rolling_game/dice_rolling_game.py`

`task2_todo_list_application/todo_list.py`

`task3_number_guessing_game/number_guessing_game.py`

`task4_atm_simulation/atm_simulation.py`

8. Testing and Verification

All Python scripts were checked with `py_compile` to confirm that the files have valid Python syntax. Core helper functions were also tested from the command line. Sample output transcripts are included in the repository under the `sample_outputs` folder.

9. Conclusion

These four tasks strengthened foundational Python development skills including loops, functions, conditionals, lists, dictionaries, input validation, randomization, and object-oriented programming. The projects are simple, interactive, and suitable for demonstrating beginner-level Python application development.